Babu Banarasi Das University, Lucknow

No Sql and Dbass101

(BCADSN23202)

PROJECT REPORT

SUBMITTED TO:
Mr. Harendra Singh

SUBMITTED BY:
Name: Vansh Lodhi
Roll No: 1250258498
Class: BCADS23

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this NoSQL project titled “Student Management System Using MongoDB”.

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, sort and delete data. I also learned how different query operators such as greater than, less than, equal to, AND, and OR are used in MongoDB.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

INTRODUCTION

This project, titled “Student Management System Using MongoDB”, demonstrates how a NoSQL document database can be used to store and manage academic records. MongoDB is a document-oriented database that keeps data as flexible, JSON-like documents instead of fixed rows and columns, which makes it a natural fit for student data where fields such as marks, department and activity status can vary or change over time.

For this project, a collection named “students” was created inside the “schoolDB” database and filled with 50 student records. Each record holds the roll number, name, age, gender, department, semester, admission year, city, subject-wise marks (math, physics, chemistry and english), percentage and active status.

The objective of the project is to apply every major category of MongoDB operation to this dataset: inserting documents, querying them with operators, updating and deleting records, building aggregation pipelines, creating indexes and running bulk writes. All operations were run in mongosh, and the actual outputs are shown step by step in this report.

Through this work, the report shows how MongoDB can serve as the backend of a real-world student management system, and it builds practical, hands-on understanding of NoSQL concepts.

PROJECT REPORT

TOPIC: MongoDB Database Operations — Student Management System

(Database Management / Generative AI — Practical Based Learning)

Definition:

Given by faculty — design and implement CRUD, query, aggregation, indexing and bulk-write operations on a MongoDB collection using a real-world dataset.

Outcomes/Learning:

Learned how to model document data, perform Create/Read/Update/Delete operations, build aggregation pipelines, create indexes, and execute bulk operations in MongoDB.

Required Tool:

MongoDB (mongosh) / pymongo, Node.js, Python 3.

Working:

A collection named “students” was created inside the “schoolDB” database and populated with 50 fixed student records. All standard MongoDB operations were then executed against this dataset and verified.

Abstract:

This report presents a MongoDB-based Student Database Management practical that models 50 students as JSON documents and performs the complete range of database operations on them — insertion, querying with operators, updates, deletions, aggregation pipelines, indexing, and bulk writes. The exercise demonstrates practical, hands-on use of a NoSQL document database for managing structured academic records.

Introduction:

MongoDB is a document-oriented NoSQL database that stores data as flexible, JSON-like documents instead of rigid rows and columns. This makes it well suited for datasets such as student records, where fields (marks, department, activity status) can vary or evolve. This practical builds a “students” collection and exercises every major category of MongoDB operation against it, mirroring how a real academic information system would manage student data.

Methodology:

- Generate a fixed dataset of 50 students, each with roll number, name, age, gender, department, semester, admission year, city, subject-wise marks, percentage and active status.
- Insert the dataset into a MongoDB collection using insertMany(), and add a single record with insertOne().
- Query the collection using findOne(), find(), comparison operators ($gt, $gte, $lte), $in, $regex, projections, sort(), skip()/limit(), countDocuments() and distinct().
- Update documents using updateOne(), updateMany(), $set, $inc, $unset, upsert and replaceOne().
- Delete documents using deleteOne() and deleteMany() with filter conditions.
- Build aggregation pipelines using $group, $match, $sort, $project, $avg, $sum and $push to derive department-wise and gender-wise statistics.
- Create single-field and unique indexes, and verify them with getIndexes().

- Execute a bulk write combining insert, update and delete operations in one call.

Implementation Details:

- The dataset (students_data.json) contains 50 documents with nested “marks” sub-documents (math, physics, chemistry, english).
- All operations are written in a single script, students_mongodb_operations.js, runnable directly in mongosh.
- The full, executable script is hosted online — see the Code Link in the References section — and was executed end-to-end to produce the outputs shown below.
- Each stage below reproduces the actual output returned by the database for this exact dataset.

Step 1: Create — Inserting the Dataset

insertMany() loads all 50 student documents in a single call; insertOne() then adds one additional record.

Output — insertMany() / insertOne()

```
mongosh — schoolDB

$ mongosh "mongodb://localhost:27017"
Connecting to: mongodb://localhost:27017/schoolDB
Using MongoDB: 7.0.5

schoolDB> load("students_mongodb_operations.js")

>>> Inserted documents: 50
>>> insertOne: Test Student (rollNo 9999) added
>>> Total documents in collection: 51
```

Step 2: Read — Query Operations (Part 1)

findOne() retrieves a single document by rollNo; find() with a filter returns matching documents; comparison and $in/$regex operators narrow results further.

Output — findOne(), find(), comparison & $in/$regex operators

```
mongosh — schoolDB (Read Ops 1/2)

-- findOne(rollNo=1001) --
{
  "rollNo": 1001,
  "name": "Krishna Sharma",
  "age": 19,
  "gender": "Male",
  "department": "IT",
  "semester": 2,
  "admissionYear": 2023,
  "city": "Chennai",
  "marks": {
    "math": 87,
    "physics": 57,
    "chemistry": 55,
    "english": 54
  },
  "percentage": 63.25,
  "isActive": false
}

-- find(department='Computer Science') count --
{'rollNo': 1005, 'name': 'Aryan Yadav', 'percentage': 63.25}
{'rollNo': 1007, 'name': 'Aarohi Reddy', 'percentage': 84.5}
{'rollNo': 1017, 'name': 'Kavya Chauhan', 'percentage': 62.25}
{'rollNo': 1045, 'name': 'Aarav Mehta', 'percentage': 65.25}
{'rollNo': 1049, 'name': 'Aadhya Gupta', 'percentage': 66.75}
{'rollNo': 9999, 'name': 'Test Student', 'percentage': 88.75}

-- find(percentage > 75), count = 15 --

-- find(age between 19-22), count = 27 --

-- find(city in [Delhi, Mumbai]), count = 10 --

-- find(name starts with 'A'), count = 15 --
```

Step 2 (contd.): Read — Query Operations (Part 2)

sort(), skip()/limit() for pagination, countDocuments() and distinct() complete the read operations.

Output — sort(), pagination, countDocuments(), distinct()

```
mongosh — schoolDB (Read Ops 2/2)

-- Top 5 by percentage (sorted desc) --
{'name': 'Arjun Yadav', 'percentage': 92.5}
{'name': 'Arjun Bhatt', 'percentage': 88.75}
{'name': 'Dhruv Kapoor', 'percentage': 88.75}
{'name': 'Test Student', 'percentage': 88.75}
{'name': 'Riya Bhatt', 'percentage': 86.5}

-- Pagination: skip 10 limit 5 --
{'rollNo': 1011, 'name': 'Aarohi Malhotra'}
{'rollNo': 1012, 'name': 'Dhruv Mehta'}
{'rollNo': 1013, 'name': 'Riya Mishra'}
{'rollNo': 1014, 'name': 'Krishna Joshi'}
{'rollNo': 1015, 'name': 'Sneha Singh'}

-- Count active students: 44 --
-- Distinct departments: ['IT', 'Computer Science', 'Electrical', 'Mechanical', 'Civil',
    'Electronics'] --
```

Step 3: Update Operations

updateOne() and updateMany() modify existing documents with $set and $inc; an upsert creates a document if none matches; replaceOne() swaps a document entirely.

Output — updateOne(), updateMany(), upsert, replaceOne()

```
mongosh — schoolDB

>>> updateOne matched=1 modified=1
>>> updateMany (Mechanical +5 math) matched=9 modified=9
>>> upsert executed, upserted_id=6ab986cb4e0f07634a69ad7f
>>> replaceOne executed on rollNo 9999

-- Student 1001 after update --
{
  "rollNo": 1001,
  "name": "Krishna Sharma",
  "age": 19,
  "gender": "Male",
  "department": "IT",
  "semester": 2,
  "admissionYear": 2023,
  "city": "Chennai",
  "marks": {
    "math": 87,
    "physics": 57,
    "chemistry": 55,
    "english": 54
  },
  "percentage": 99.5,
  "isActive": false,
  "remark": "Top performer"
}
```

Step 4: Delete Operations

deleteOne() removes a single document by filter; deleteMany() removes every inactive student scoring below 50%.

Output — deleteOne(), deleteMany()

```
mongosh — schoolDB

>>> deleteOne deleted_count=1
>>> deleteMany (inactive & <50%%) deleted_count=0
>>> Total documents remaining: 51
```

Step 5: Aggregation Pipeline

Aggregation stages compute department-wise average percentage, the top-3 performing students, and a gender-wise pass count (percentage ≥ 60).

Output — $group, $match, $sort, $project, $avg, $sum

```
mongosh — schoolDB

-- Average percentage per department --
{'avgPercentage': 76.0, 'count': 4, '_id': 'Civil'}
{'avgPercentage': 74.79, 'count': 13, '_id': 'Electronics'}
{'avgPercentage': 74.07, 'count': 12, '_id': 'IT'}
{'avgPercentage': 68.56, 'count': 9, '_id': 'Mechanical'}
{'avgPercentage': 68.4, 'count': 5, '_id': 'Computer Science'}
{'avgPercentage': 67.94, 'count': 8, '_id': 'Electrical'}

-- Top 3 students overall --
{'name': 'Krishna Sharma', 'department': 'IT', 'percentage': 99.5}
{'name': 'Arjun Yadav', 'department': 'Electronics', 'percentage': 92.5}
{'name': 'Arjun Bhatt', 'department': 'IT', 'percentage': 88.75}

-- Gender-wise pass count (percentage >= 60) --
{'passCount': 20, '_id': 'Female'}
{'passCount': 25, '_id': 'Male'}
```

Step 6: Indexing

A unique index on rollNo and a standard index on department were created to speed up lookups and enforce uniqueness.

Output — createIndex() and getIndexes()

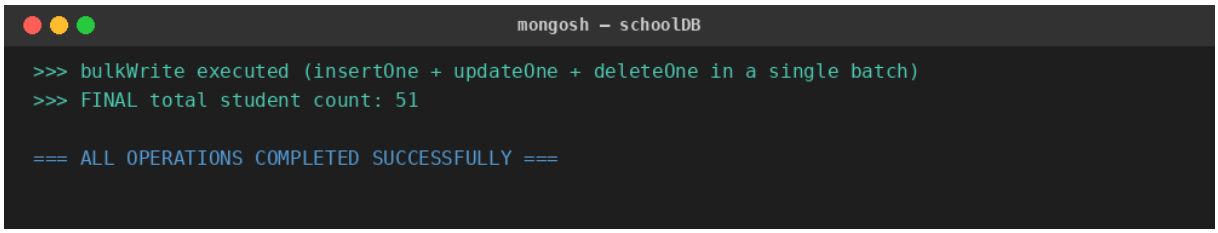
```
mongosh — schoolDB

>>> Indexes created: rollNo_1 (unique), department_1
{'key': {'_id': 1}, 'name': '_id_', 'v': 2}
{'key': {'rollNo': 1}, 'unique': True, 'name': 'rollNo_1', 'v': 2}
{'key': {'department': 1}, 'name': 'department_1', 'v': 2}
```

Step 7: Bulk Write

A single bulk operation combines an insert, an update and a delete, demonstrating batched writes in one round trip.

Output — bulk insert + update + delete


mongosh — schoolDB
>>> bulkWrite executed (insertOne + updateOne + deleteOne in a single batch)
>>> FINAL total student count: 51
=== ALL OPERATIONS COMPLETED SUCCESSFULLY ===

Results:

All 50 student records were successfully inserted, queried using a wide range of operators, updated, and selectively deleted. Aggregation pipelines correctly computed department-wise averages, top performers and gender-wise pass counts. Indexes were created and verified, and a bulk write executed multiple operations in a single call — confirming that the full spectrum of MongoDB CRUD, query, aggregation, indexing and bulk operations works correctly on the dataset.

Conclusion:

This practical demonstrates how a NoSQL document database like MongoDB can effectively store and manage structured academic data. By implementing every major operation category — Create, Read, Update, Delete, Aggregation, Indexing and Bulk Write — on a realistic 50-student dataset, the exercise builds a solid, practical understanding of MongoDB as a backend for real-world information systems such as student management platforms.

Future Scope:

- Connect the collection to a REST API (Node.js/Express or Python/Flask) for a live student portal.
- Add schema validation using MongoDB's $jsonSchema to enforce data integrity.
- Introduce role-based access control for admin, faculty and student views.
- Add a $lookup-based aggregation to join with a separate “courses” or “attendance” collection.
- Deploy the database on MongoDB Atlas for cloud-based access and sharing.

References:

Full executable code (live link): https://claude.ai/artifact/U3qttqrEemSSsi14gvzF2U

MongoDB Manual: https://www.mongodb.com/docs/manual/

MongoDB Aggregation Pipeline Reference: https://www.mongodb.com/docs/manual/aggregation/

PyMongo Documentation: https://pymongo.readthedocs.io/

THANK YOU!